

DOCUMENTACION COMPLETA DEL PROYECTO FLASHCARDS

Generado: 2026-02-04 12:15

INDICE 1. Vision general 2. Estructura de archivos 3. Diccionario de nombres (componentes, elementos y variables) 4. index.html (estructura y UI) 5. styles.css (tema visual y layout) 6. flashcards.js (logica completa y detallada) 7. Codigo fuente completo (referencia)

1. Vision general

Este proyecto es una app web estatica de flashcards. Carga preguntas desde Google Sheets en CSV, y si no esta disponible usa data.json. Permite filtrar por categorias, barajar tarjetas, y registrar progreso local con un modo inteligente de aleatoriedad.

2. Estructura de archivos

- index.html
- styles.css
- flashcards.js

3. Diccionario de nombres (componentes, elementos y variables)

3.1 HTML / UI

- container: contenedor principal que centra y limita el ancho de la app.
- layout: grid principal con sidebar a la izquierda y main a la derecha.
- sidebar: columna lateral que contiene categorias y opciones.
- panel: bloque desplegable para categorias y opciones.
- panel-title: titulo clickable del panel desplegable.
- panel-body: contenido interno del panel.
- sheetTabs: contenedor de tabs si hay multiples hojas.
- flashcard: tarjeta interactiva (canvas visual de la flashcard).
- front/back: caras frontal y trasera de la tarjeta.
- nextBtn: boton para siguiente tarjeta.
- categorySelect: selector multiple de categorias.
- modeSelect: selector de modo (inteligente / clasico).
- randomnessSelect: selector de nivel de aleatoriedad.
- avoidRepeat: checkbox para evitar repeticion reciente.
- markCorrect / markWrong: botones para registrar acierto o fallo.
- resetProgress: boton para reiniciar progreso.
- statsText: texto con resumen de estadisticas.

3.2 JavaScript (datos y estado)

- flashcardsData: lista activa de tarjetas tras filtrar categorias.
- originalData: lista completa de tarjetas cargadas.
- currentIndex: indice actual dentro de flashcardsData.
- selectedCategories: conjunto de categorias seleccionadas.
- recentIds: cola de ids recientes para evitar repeticion.
- progress: objeto con estadisticas por tarjeta (correct, wrong, seen).
- settings: opciones persistentes (mode, randomness, avoidRepeat).
- STORAGE_PROGRESS_KEY / STORAGE_SETTINGS_KEY: claves de localStorage.

3.3 JavaScript (funciones)

- initFlashcards: inicializa datos, ids, UI y estados.
- shuffleFlashcards: baraja tarjetas con Fisher-Yates.
- parseCSV / parseCSVLine: parsean CSV a objetos.
- showFlashcard: renderiza la tarjeta actual y actualiza stats.
- toggleFlip: voltea la tarjeta.
- nextFlashcard: calcula la siguiente tarjeta y la muestra.
- initCategories / handleCategorySelection / filterByCategories: gestion de categorias.
- updateNextButtonText: actualiza contador del boton siguiente.
- markAnswer: registra un acierto o fallo y avanza.
- pickNextIndex: selecciona el siguiente indice con peso y aleatoriedad.
- calcWeight: calcula el peso de cada tarjeta segun fallos/aciertos.
- getRandomnessJitter: ajusta variacion segun modo.
- getRecentWindow / rememberRecent: evita repeticion reciente.
- updateStatsUI: resume estadisticas del set actual.
- registerSeen: incrementa visualizaciones.
- createCardId: genera un id hash estable para cada tarjeta.
- load/saveProgress / load/saveSettings: persistencia en localStorage.
- applySettingsToUI: aplica settings al UI.

4. index.html (estructura y UI)

- Contiene el layout principal con sidebar (categorias y opciones) y el contenido central con la flashcard.
- Usa <details> para secciones desplegables de categorias y opciones.
- Incluye controles de modo, aleatoriedad, evitar repeticion, botones Bien/Mal y estadisticas.
- Define el contenedor de flashcard con front y back para el flip 3D.

5. styles.css (tema visual y layout)

- Define variables CSS para colores, sombras, radios y medidas.
- Layout con .layout: sidebar izquierda y main a la derecha.
- Paneles desplegables con estilos para <details>/<summary>.
- Estilos de select, botones, cards, y animacion de flip 3D.
- Media queries para responsivo (<900px y <480px).

6. flashcards.js (logica completa y detallada)

6.1 Variables y estado inicial

- Se declaran estructuras globales para datos (flashcardsData, originalData), estado (currentIndex, selectedCategories, recentIds) y persistencia (progress, settings).
- progress y settings se cargan desde localStorage al iniciar.

6.2 Carga de datos

- Se intenta obtener un CSV de Google Sheets mediante fetch.
- parseCSV convierte el CSV en objetos {question, answer, category,...}.
- Si falla, se carga data.json local.

6.3 Inicializacion de tarjetas

- initFlashcards filtra filas vacias y genera un id estable por tarjeta.
- Si no existe progreso previo para una tarjeta, se inicializa con 0.
- Se inicializa UI de categorias, se baraja y se aplica el estado inicial a la UI.

6.4 Render y eventos

- showFlashcard renderiza la tarjeta actual y registra que fue vista.
- toggleFlip aplica el flip 3D en la tarjeta (click o teclado).
- nextFlashcard usa pickNextIndex para elegir la siguiente tarjeta.

6.5 Gestion de categorias

- initCategories crea el select multiple y agrega la opcion Todas.
- handleCategorySelection asegura la logica de 'Todas' vs multiples.
- filterByCategories aplica el filtro y actualiza el contador del boton.

6.6 Progreso y controles

- markAnswer actualiza stats (correct/wrong) y avanza.
- resetProgress limpia el progreso y recalcula stats visibles.
- Las opciones del menu guardan settings en localStorage.

6.7 Aleatoriedad inteligente

- pickNextIndex construye una lista de candidatos y calcula un peso por tarjeta.
- Si avoidRepeat esta activo, excluye ids recientes en recentIds.
- calcWeight aumenta el peso cuando hay mas fallos que aciertos.
- getRandomnessJitter aplica un factor aleatorio segun nivel.
- getRecentWindow define cuantos ids recordar para evitar repeticiones.

6.8 Estadisticas

- updateStatsUI calcula totales del set filtrado y el score actual.
- registerSeen incrementa las visualizaciones por tarjeta.

6.9 Persistencia

- createCardId crea un hash con question/answer/category para id estable.
- load/saveProgress y load/saveSettings manejan localStorage.
- applySettingsToUI sincroniza la UI con la configuracion guardada.

7.Codigo fuente completo (referencia)

```
--- index.html ---
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flashcards</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="container">
    <div class="layout">
      <aside class="sidebar">
        <details class="panel" open>
```

```

<summary class="panel-title">Categorias</summary>
<div class="panel-body">
  <label for="categorySelect" class="category-label">Selecciona categorias
(multiple):</label>
  <select id="categorySelect" aria-label="Seleccionar categorias" class="category-
select"></select>
  <small class="category-hint">Manten Ctrl/Cmd para seleccionar multiples
categorias</small>
</div>
</details>

<details class="panel" open>
<summary class="panel-title">Opciones</summary>
<div class="panel-body">
  <div class="menu">
    <div class="menu-row">
      <div class="menu-block">
        <label class="menu-label" for="modeSelect">Modo</label>
        <select id="modeSelect" class="menu-select">
          <option value="smart">Aleatorio inteligente</option>
          <option value="random">Aleatorio clasico</option>
        </select>
      </div>
      <div class="menu-block">
        <label class="menu-label" for="randomnessSelect">Aleatoriedad</label>
        <select id="randomnessSelect" class="menu-select">
          <option value="soft">Suave</option>
          <option value="normal">Normal</option>
          <option value="chaos">Caos</option>
        </select>
      </div>
      <label class="menu-check">
        <input type="checkbox" id="avoidRepeat" checked>
        Evitar repeticion reciente
      </label>
    </div>
    <div class="menu-row">
      <button id="markCorrect" class="button-secondary good"
type="button">Bien</button>
      <button id="markWrong" class="button-secondary bad" type="button">Mal</button>
      <button id="resetProgress" class="button-ghost" type="button">Reiniciar
progreso</button>
    </div>
    <div class="stats">
      <span id="statsText">Tarjetas: 0</span>
    </div>
  </div>
</details>

```

```

    </details>
  </aside>

  <main class="main">
    <div class="sheets-container">
      <div id="sheetTabs" class="sheet-tabs"></div>
    </div>

    <div class="flashcard" id="flashcard" role="button" aria-pressed="false" tabindex="0">
      <div class="flashcard-inner">
        <div class="front">Haz clic para ver la respuesta</div>
        <div class="back">Respuesta</div>
      </div>
    </div>

    <button id="nextBtn" class="button-next" type="button">Siguiente</button>
  </main>
</div>
</div>

<script src="flashcards.js"></script>
</body>
</html>

```

--- styles.css ---

```

:root{
  --color-background: #202020;
  --color-flashcard-dark: #18181b;
  --color-flashcard-light: #2c2c31;
  --color-primary: #3b7f4a;
  --color-primary-hover: #49a05c;
  --color-text-light: #f7f7f8;
  --color-text-muted: #aaaaae;
  --color-border-subtle: rgba(255,255,255,0.08);
  --color-good: #2f7d4c;
  --color-bad: #a34a4a;
  --border-radius-card: 16px;
  --box-shadow-card: 0 16px 40px rgba(0,0,0,0.65);
  --box-shadow-focus: 0 0 0 4px rgba(59,127,74,0.3);
  --container-gap: 30px;
  --max-width: 1040px;
}

html, body {
  margin: 0;
  padding: 0;
  height: 100%;
  font-family: "Inter", system-ui, -apple-system, sans-serif;

```

```
background: var(--color-background);
color: var(--color-text-light);
-webkit-font-smoothing: antialiased;
box-sizing: border-box;
}
*, ::before, ::after { box-sizing: inherit; }
```

```
.container {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: stretch;
  justify-content: flex-start;
  gap: var(--container-gap);
  padding: 16px 28px 28px;
  width: 100%;
  max-width: var(--max-width);
  margin: 0 auto;
}
```

```
.layout {
  display: flex;
  gap: 24px;
  width: 100%;
  align-items: flex-start;
}
```

```
.sidebar {
  width: 280px;
  min-width: 240px;
  display: flex;
  flex-direction: column;
  gap: 16px;
}
```

```
.main {
  flex: 1;
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: var(--container-gap);
}
```

```
.panel {
  border: 1px solid var(--color-border-subtle);
  border-radius: 14px;
  background: linear-gradient(160deg, rgba(36,36,42,0.9), rgba(24,24,28,0.9));
  box-shadow: 0 12px 28px rgba(0,0,0,0.45);
}
```

```
    overflow: hidden;
}

.panel-title {
  list-style: none;
  cursor: pointer;
  padding: 14px 16px;
  font-weight: 600;
  font-size: 14px;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  color: var(--color-text-light);
  background: rgba(255,255,255,0.03);
  border-bottom: 1px solid var(--color-border-subtle);
}

.panel-title::-webkit-details-marker {
  display: none;
}

.panel-title::after {
  content: "?";
  float: right;
  opacity: 0.7;
  transition: transform 0.2s ease;
}

.panel[open] .panel-title::after {
  transform: rotate(180deg);
}

.panel-body {
  padding: 14px 16px 16px;
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.category-container {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 8px;
  text-align: center;
}

.category-label {
  font-size: 13px;
```

```
    font-weight: 500;
    color: var(--color-text-muted);
}

.category-hint {
    font-size: 12px;
    color: var(--color-text-muted);
    opacity: 0.7;
}

.category-select {
    appearance: none;
    padding: 12px 14px;
    font-size: 15px;
    border-radius: 12px;
    background-color: var(--color-flashcard-light);
    color: var(--color-text-light);
    border: 1px solid var(--color-border-subtle);
    cursor: pointer;
    width: 100%;
    text-align: left;
    transition: background-color .2s ease, box-shadow .2s ease;
    box-shadow: 0 6px 16px rgba(0,0,0,0.4);
}

.category-select:hover{
    background-color: #38383e;
    border-color: rgba(255,255,255,0.15);
}

.category-select:focus{
    outline: none;
    box-shadow: var(--box-shadow-focus);
    overflow: auto;
}

.category-select[multiple] {
    height: 200px;
    min-height: 120px;
    text-align: left;
    padding: 8px;
    scrollbar-width: none; /* Firefox */
    -ms-overflow-style: none; /* IE/Edge */
}

.category-select[multiple] option {
    padding: 8px 12px;
    margin: 2px 0;
}
```



```
border-radius: 6px;
cursor: pointer;
transition: background-color 0.2s ease;
}

.category-select[multiple] option:hover,
.category-select[multiple] option:checked {
  background-color: var(--color-primary);
  color: var(--color-text-light);
}

.menu {
  width: 100%;
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.menu-row {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  align-items: center;
  justify-content: flex-start;
}

.menu-block {
  display: flex;
  flex-direction: column;
  gap: 6px;
  min-width: 180px;
  width: 100%;
}

.menu-label {
  font-size: 12px;
  font-weight: 600;
  color: var(--color-text-muted);
  text-transform: uppercase;
  letter-spacing: 0.08em;
}

.menu-select {
  appearance: none;
  padding: 10px 14px;
  font-size: 14px;
  border-radius: 10px;
  background-color: #2b2b31;
```

```
    color: var(--color-text-light);
    border: 1px solid var(--color-border-subtle);
    cursor: pointer;
    transition: background-color .2s ease, box-shadow .2s ease;
    box-shadow: 0 6px 16px rgba(0,0,0,0.3);
    width: 100%;
}

.menu-select:hover {
    background-color: #38383e;
}

.menu-select:focus {
    outline: none;
    box-shadow: var(--box-shadow-focus);
}

.menu-check {
    display: flex;
    align-items: center;
    gap: 8px;
    font-size: 13px;
    color: var(--color-text-light);
    background: rgba(255,255,255,0.04);
    padding: 10px 12px;
    border-radius: 10px;
    border: 1px solid var(--color-border-subtle);
    width: 100%;
}

.menu-check input {
    accent-color: var(--color-primary);
}

.stats {
    font-size: 12px;
    color: var(--color-text-muted);
    text-align: left;
    letter-spacing: 0.02em;
}

.button-secondary,
.button-ghost {
    padding: 12px 18px;
    font-size: 14px;
    font-weight: 600;
    border-radius: 10px;
    border: 1px solid var(--color-border-subtle);
```

```
    cursor: pointer;
    transition: transform .15s, box-shadow .15s, background-color .2s;
}

.button-secondary {
    background: #2b2b31;
    color: var(--color-text-light);
    box-shadow: 0 8px 18px rgba(0,0,0,0.35);
}

.button-secondary.good {
    background: rgba(47,125,76,0.25);
    border-color: rgba(47,125,76,0.6);
}

.button-secondary.bad {
    background: rgba(163,74,74,0.25);
    border-color: rgba(163,74,74,0.6);
}

.button-secondary:hover {
    transform: translateY(-2px);
}

.button-ghost {
    background: transparent;
    color: var(--color-text-muted);
    border-style: dashed;
}

.button-ghost:hover {
    color: var(--color-text-light);
    border-color: rgba(255,255,255,0.2);
}

.flashcard {
    width: 380px;
    max-width: 90vw;
    height: 250px;
    perspective: 1200px;
    display: flex;
    align-items: center;
    justify-content: center;
    cursor: pointer;
    -webkit-tap-highlight-color: transparent;
}

.flashcard-inner {
```

```

width: 100%;
height: 100%;
position: relative;
transform-style: preserve-3d;
transition: transform 0.7s cubic-bezier(.25,.8,.25,1);
}

.flashcard.flipped .flashcard-inner { transform: rotateY(180deg); }

.front, .back {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  background: var(--color-flashcard-dark);
  border-radius: var(--border-radius-card);
  box-shadow: var(--box-shadow-card);
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 30px;
  text-align: center;
  font-weight: 500;
  color: var(--color-text-light);
  backface-visibility: hidden;
  border: 1px solid var(--color-border-subtle);
  line-height: 1.4;
}

.back { transform: rotateY(180deg); }

.front {
  background: linear-gradient(145deg, #222226, var(--color-flashcard-dark));
  color: var(--color-text-muted);
  font-size: 1.15rem;
}

.back {
  background: linear-gradient(145deg, var(--color-flashcard-dark), #2a2a2d);
  font-size: 1.25rem;
  font-weight: 600;
}

.button-next {
  padding: 14px 30px;
  font-size: 16px;
  font-weight: 600;
  border-radius: 12px;

```

```
background: var(--color-primary);
color: var(--color-text-light);
border: none;
cursor: pointer;
box-shadow: 0 10px 25px rgba(0,0,0,0.4);
transition: transform .15s, box-shadow .15s, background-color .18s;
letter-spacing: 0.5px;
}

.button-next:hover {
  background: var(--color-primary-hover);
  transform: translateY(-3px);
  box-shadow: 0 12px 30px rgba(0,0,0,0.5);
}

.button-next:active {
  transform: translateY(0);
  box-shadow: 0 6px 15px rgba(0,0,0,0.4);
}

.button-next:focus {
  outline: none;
  box-shadow: var(--box-shadow-focus);
}

.button-next:disabled {
  background-color: #666;
  cursor: not-allowed;
  transform: none;
  box-shadow: 0 6px 15px rgba(0,0,0,0.3);
}

.button-next:disabled:hover {
  background-color: #666;
  transform: none;
  box-shadow: 0 6px 15px rgba(0,0,0,0.3);
}

.sheets-container {
  width: 100%;
  display: flex;
  justify-content: center;
  margin-bottom: 10px;
}

.sheet-tabs {
  display: flex;
  gap: 8px;
```

```
flex-wrap: wrap;
justify-content: center;
max-width: 100%;
}
```

```
.sheet-tab {
padding: 10px 16px;
background: var(--color-flashcard-light);
color: var(--color-text-light);
border: 1px solid var(--color-border-subtle);
border-radius: 8px;
cursor: pointer;
transition: all 0.2s ease;
font-size: 14px;
font-weight: 500;
}
```

```
.sheet-tab:hover {
background: #38383e;
transform: translateY(-1px);
}
```

```
.sheet-tab.active {
background: var(--color-primary);
border-color: var(--color-primary);
}
```

```
@media (max-width: 900px) {
.layout {
flex-direction: column;
align-items: stretch;
}

.sidebar {
width: 100%;
min-width: 0;
}

.main {
align-items: center;
}
}
```

```
@media (max-width: 480px) {
.sheet-tabs {
gap: 6px;
}
}
```

```
.sheet-tab {  
  padding: 8px 12px;  
  font-size: 13px;  
}  
}  
  
@media (max-width: 480px) {  
  .flashcard {  
    width: 100%;  
    max-width: calc(100vw - 40px);  
    height: 220px;  
  }  
  
  .container {  
    padding: 20px;  
    gap: 24px;  
  }  
  
  .category-select,  
  .category-select[multiple] {  
    width: 100%;  
    max-width: 350px;  
    padding: 10px 14px;  
  }  
  
  .menu {  
    padding: 0;  
  }  
  
  .menu-row {  
    flex-direction: column;  
    align-items: stretch;  
  }  
  
  .menu-block {  
    min-width: 100%;  
  }  
  
  .menu-check {  
    justify-content: center;  
  }  
  
  .button-next,  
  .button-secondary,  
  .button-ghost {  
    width: 100%;  
    max-width: 350px;  
  }  
}
```

```

    .front, .back {
        font-size: 1rem;
        padding: 20px;
    }

    .back { font-size: 1.1rem; }

    .category-label { font-size: 13px; }
    .category-hint { font-size: 11px; }
}

--- flashcards.js ---
let flashcardsData = [];
let originalData = []; // Guardamos todas las flashcards
let currentIndex = 0;
let selectedCategories = new Set(); // Para almacenar categorias seleccionadas
let recentIds = [];

const STORAGE_PROGRESS_KEY = "flashcards_progress_v1";
const STORAGE_SETTINGS_KEY = "flashcards_settings_v1";

let progress = loadProgress();
let settings = loadSettings();

// =====
// FUNCION PARA INICIALIZAR LA WEB
function initFlashcards(data) {
    // Filtramos filas vacias o mal formadas
    const cleanData = data.filter(card => card.question && card.answer);
    cleanData.forEach(card => {
        card._id = createCardId(card);
        if (!progress[card._id]) {
            progress[card._id] = { correct: 0, wrong: 0, seen: 0 };
        }
    });
};

flashcardsData = cleanData;
originalData = cleanData;

initCategories();
shuffleFlashcards(); // Barajar al inicializar
applySettingsToUI();
updateNextButtonText();
showFlashcard();
}

// =====

```



```

// BARAJAR FLASHCARDS (algoritmo Fisher-Yates)
function shuffleFlashcards() {
  for (let i = flashcardsData.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [flashcardsData[i], flashcardsData[j]] = [flashcardsData[j], flashcardsData[i]];
  }
  currentIndex = 0; // Reiniciar indice despues de barajar
}

// =====
// PARSEADOR CSV robusto: soporta comillas, comas internas y celdas vacias
function parseCSV(csvText) {
  // Quitar BOM si existe
  if (csvText.charCodeAt(0) === 0xFEFF) csvText = csvText.slice(1);

  const lines = csvText.split(/\r?\n/).filter(line => line.trim() !== "");
  if (!lines.length) return [];

  // Parse headers usando la misma funcion de linea
  const headers = parseCSVLine(lines[0]).map(h => h.trim());

  const rows = lines.slice(1).map(line => {
    const cols = parseCSVLine(line);
    const obj = {};
    headers.forEach((h, i) => {
      obj[h] = (cols[i] || "").trim();
    });
    return obj;
  });

  return rows;
}

// parsea una linea CSV respetando comillas dobles y comas internas
function parseCSVLine(line) {
  const result = [];
  let cur = "";
  let insideQuotes = false;
  for (let i = 0; i < line.length; i++) {
    const ch = line[i];

    if (ch === '"' ) {
      // si es una comilla y la siguiente tambien es comilla -> escapada -> anadimos una
      // comilla
      if (insideQuotes && line[i+1] === '"') {
        cur += '"';
        i++; // saltar la comilla escapada
      } else {

```

```

        insideQuotes = !insideQuotes;
    }
} else if (ch === ',' && !insideQuotes) {
    result.push(cur);
    cur = "";
} else {
    cur += ch;
}
}
result.push(cur);
return result;
}

// =====
// CARGA DESDE GOOGLE SHEETS (CSV) con fallback a JSON local
fetch("https://docs.google.com/spreadsheets/d/e/2PACX-1vSNGZwo-97c1vhJdxEzrS4-
RBL5PJuoPu_KGw5gdaTYIO61YwgkB76YSeDmuOKFXr7o9y_41LLYMFAf/pub?gid=0&single=true&output=csv")
    .then(res => res.text())
    .then(csv => {
        const parsed = parseCSV(csv);
        initFlashcards(parsed);
    })
    .catch(() => {
        console.warn("Google Sheets no disponible. Usando JSON local...");

        fetch("data.json")
            .then(res => res.json())
            .then(data => initFlashcards(data))
            .catch(err => console.error("Error al cargar JSON de fallback:", err));
    });

// =====
// ELEMENTOS DEL DOM
const flashcardEl = document.getElementById('flashcard');
const frontEl = flashcardEl.querySelector('.front');
const backEl = flashcardEl.querySelector('.back');
const nextBtn = document.getElementById('nextBtn');
const categorySelect = document.getElementById('categorySelect');
const modeSelect = document.getElementById('modeSelect');
const randomnessSelect = document.getElementById('randomnessSelect');
const avoidRepeatCheckbox = document.getElementById('avoidRepeat');
const markCorrectBtn = document.getElementById('markCorrect');
const markWrongBtn = document.getElementById('markWrong');
const resetProgressBtn = document.getElementById('resetProgress');
const statsText = document.getElementById('statsText');

// =====
// MOSTRAR TARJETA ACTUAL

```

```

function showFlashcard() {
  if (!flashcardsData.length) {
    frontEl.textContent = "No hay tarjetas en las categorias seleccionadas";
    backEl.textContent = "";
    updateStatsUI();
    return;
  }
  const card = flashcardsData[currentIndex];
  frontEl.textContent = card.question;
  backEl.textContent = card.answer;
  flashcardEl.classList.remove('flipped');

  registerSeen(card);
  rememberRecent(card._id);
  updateStatsUI();
}

// =====
// VOLTEAR TARJETA
flashcardEl.addEventListener('click', () => toggleFlip());
flashcardEl.addEventListener('keydown', (e) => {
  if (e.key === 'Enter' || e.key === ' ') {
    e.preventDefault();
    toggleFlip();
  }
});

function toggleFlip() {
  flashcardEl.classList.toggle('flipped');
}

// =====
// SIGUIENTE TARJETA (ALEATORIA)
nextBtn.addEventListener('click', nextFlashcard);
function nextFlashcard() {
  if (!flashcardsData.length) return;
  currentIndex = pickNextIndex();
  showFlashcard();
}

// =====
// CATEGORIAS AUTOMATICAS
function initCategories() {
  // filtrar categorias validas (no undefined ni vacias)
  const cats = originalData.map(card => card.category).filter(Boolean);
  const categories = ["Todas", ...new Set(cats)];

  // limpiar select si ya tiene opciones

```

```

categorySelect.innerHTML = "";

// Hacer el select multiple
categorySelect.setAttribute('multiple', 'true');
categorySelect.size = Math.min(6, categories.length); // Mostrar maximo 6 opciones a la vez

categories.forEach(cat => {
  const opt = document.createElement("option");
  opt.value = cat === "Todas" ? "all" : cat;
  opt.textContent = cat;
  categorySelect.appendChild(opt);
});

// Seleccionar "Todas" por defecto
selectedCategories.add("all");
categorySelect.querySelector('option[value="all"]').selected = true;

categorySelect.addEventListener("change", handleCategorySelection);
}

// =====
// MANEJAR SELECCION MULTIPLE DE CATEGORIAS
function handleCategorySelection() {
  const selectedOptions = Array.from(categorySelect.selectedOptions);
  const selectedValues = selectedOptions.map(opt => opt.value);

  // Manejar logica de seleccion
  if (selectedValues.includes("all")) {
    // Si se selecciona "Todas", deseleccionar todo lo demas
    selectedCategories.clear();
    selectedCategories.add("all");
    Array.from(categorySelect.options).forEach(opt => {
      opt.selected = opt.value === "all";
    });
  } else {
    // Si se selecciona cualquier otra categoria, quitar "Todas"
    selectedCategories.delete("all");
    if (selectedValues.length === 0) {
      // Si no hay nada seleccionado, seleccionar "Todas" por defecto
      selectedCategories.add("all");
      categorySelect.querySelector('option[value="all"]').selected = true;
    } else {
      // Agregar las categorias seleccionadas
      selectedCategories.clear();
      selectedValues.forEach(val => selectedCategories.add(val));
    }
  }
}

```

```

    filterByCategories();
}

// =====
// FILTRAR POR MULTIPLES CATEGORIAS
function filterByCategories() {
  if (selectedCategories.has("all")) {
    flashcardsData = [...originalData];
  } else {
    flashcardsData = originalData.filter(c =>
      c.category && selectedCategories.has(c.category)
    );
  }
  shuffleFlashcards(); // Barajar al cambiar categorias
  showFlashcard();

  // Actualizar texto del boton para mostrar cantidad
  updateNextButtonText();
}

// =====
// ACTUALIZAR TEXTO DEL BOTON SIGUIENTE
function updateNextButtonText() {
  const count = flashcardsData.length;
  if (count === 0) {
    nextBtn.textContent = "Siguiente (0)";
    nextBtn.disabled = true;
  } else {
    nextBtn.textContent = `Siguiente (${count})`;
    nextBtn.disabled = false;
  }
}

// =====
// CONTROLES DE PROGRESO Y MODO
modeSelect.addEventListener('change', () => {
  settings.mode = modeSelect.value;
  saveSettings();
});

randomnessSelect.addEventListener('change', () => {
  settings.randomness = randomnessSelect.value;
  saveSettings();
});

avoidRepeatCheckbox.addEventListener('change', () => {
  settings.avoidRepeat = avoidRepeatCheckbox.checked;
  saveSettings();
});

```

```

});

markCorrectBtn.addEventListener('click', () => markAnswer(true));
markWrongBtn.addEventListener('click', () => markAnswer(false));

resetProgressBtn.addEventListener('click', () => {
  progress = {};
  originalData.forEach(card => {
    progress[card._id] = { correct: 0, wrong: 0, seen: 0 };
  });
  saveProgress();
  updateStatsUI();
});

function markAnswer(isCorrect) {
  if (!flashcardsData.length) return;
  const card = flashcardsData[currentIndex];
  const stats = progress[card._id];
  if (isCorrect) {
    stats.correct += 1;
  } else {
    stats.wrong += 1;
  }
  saveProgress();
  updateStatsUI();
  nextFlashcard();
}

// =====
// LOGICA DE ALEATORIEDAD INTELIGENTE
function pickNextIndex() {
  if (flashcardsData.length <= 1) return 0;

  const recentWindow = getRecentWindow();
  let candidates = flashcardsData.map((card, index) => ({ card, index }));

  if (settings.avoidRepeat && recentWindow > 0) {
    candidates = candidates.filter(c => !recentIds.includes(c.card._id));
    if (!candidates.length) {
      candidates = flashcardsData.map((card, index) => ({ card, index }));
    }
  }

  if (settings.mode === 'random') {
    const randomPick = Math.floor(Math.random() * candidates.length);
    return candidates[randomPick].index;
  }
}

```

```

const weights = candidates.map(c => calcWeight(c.card));
const total = weights.reduce((sum, w) => sum + w, 0);
let r = Math.random() * total;
for (let i = 0; i < candidates.length; i++) {
  r -= weights[i];
  if (r <= 0) return candidates[i].index;
}
return candidates[candidates.length - 1].index;
}

function calcWeight(card) {
  const stats = progress[card._id] || { correct: 0, wrong: 0, seen: 0 };
  const score = stats.wrong - stats.correct;

  let weight = 1 + Math.max(0, score) * 3 + stats.wrong * 1.2;
  if (stats.seen === 0) weight += 2;

  const jitter = getRandomnessJitter();
  weight = Math.max(0.2, weight * jitter);
  return weight;
}

function getRandomnessJitter() {
  if (settings.randomness === 'soft') {
    return 1 + (Math.random() * 0.3 - 0.15);
  }
  if (settings.randomness === 'chaos') {
    return 1 + (Math.random() * 1.2 - 0.6);
  }
  return 1 + (Math.random() * 0.6 - 0.3);
}

function getRecentWindow() {
  if (!settings.avoidRepeat) return 0;
  if (settings.randomness === 'soft') return 2;
  if (settings.randomness === 'chaos') return 5;
  return 3;
}

function rememberRecent(id) {
  const limit = getRecentWindow();
  if (limit <= 0) return;
  recentIds = recentIds.filter(x => x !== id);
  recentIds.push(id);
  if (recentIds.length > limit) {
    recentIds = recentIds.slice(recentIds.length - limit);
  }
}

```

```

// =====
// ESTADISTICAS
function updateStatsUI() {
  if (!flashcardsData.length) {
    statsText.textContent = "Sin tarjetas para mostrar";
    return;
  }

  const card = flashcardsData[currentIndex];
  const stats = progress[card._id] || { correct: 0, wrong: 0, seen: 0 };
  const score = stats.wrong - stats.correct;

  const totals = flashcardsData.reduce((acc, c) => {
    const s = progress[c._id] || { correct: 0, wrong: 0, seen: 0 };
    acc.correct += s.correct;
    acc.wrong += s.wrong;
    acc.seen += s.seen;
    return acc;
  }, { correct: 0, wrong: 0, seen: 0 });

  statsText.textContent = `Tarjetas: ${flashcardsData.length} | Vistas: ${totals.seen} |
  Aciertos: ${totals.correct} | Fallos: ${totals.wrong} | Actual score: ${score}`;
}

function registerSeen(card) {
  const stats = progress[card._id];
  stats.seen += 1;
  saveProgress();
}

// =====
// STORAGE
function createCardId(card) {
  const raw = `${card.question}||${card.answer}||${card.category || ''}`;
  let hash = 0;
  for (let i = 0; i < raw.length; i++) {
    hash = ((hash << 5) - hash) + raw.charCodeAt(i);
    hash |= 0;
  }
  return `card_${Math.abs(hash)}`;
}

function loadProgress() {
  try {
    const raw = localStorage.getItem(STORAGE_PROGRESS_KEY);
    return raw ? JSON.parse(raw) : {};
  } catch (e) {

```



```

    return {};
  }
}

function saveProgress() {
  localStorage.setItem(STORAGE_PROGRESS_KEY, JSON.stringify(progress));
}

function loadSettings() {
  const defaults = { mode: 'smart', randomness: 'normal', avoidRepeat: true };
  try {
    const raw = localStorage.getItem(STORAGE_SETTINGS_KEY);
    if (!raw) return defaults;
    const parsed = JSON.parse(raw);
    return {
      mode: parsed.mode === 'random' ? 'random' : 'smart',
      randomness: ['soft', 'normal', 'chaos'].includes(parsed.randomness) ? parsed.randomness :
'normal',
      avoidRepeat: typeof parsed.avoidRepeat === 'boolean' ? parsed.avoidRepeat : true
    };
  } catch (e) {
    return defaults;
  }
}

function saveSettings() {
  localStorage.setItem(STORAGE_SETTINGS_KEY, JSON.stringify(settings));
}

function applySettingsToUI() {
  modeSelect.value = settings.mode;
  randomnessSelect.value = settings.randomness;
  avoidRepeatCheckbox.checked = settings.avoidRepeat;
}

```